

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 4

Comparative Study of Deep Convolutional Neural Network Architectures Using Transfer Learning

Name	Vinay Sathish
Roll Number	24011101126
Degree & Branch	B.Tech Artificial Intelligence & Data Science
Semester	V
Subject Code & Name	CS3807 – Deep Learning Laboratory
Academic Year	2026–27

1. Objective

The objectives of this experiment are:

- Study the evolution of deep CNN architectures.
- Compare LeNet-5, AlexNet, VGG16, GoogleNet and ResNet.
- Understand transfer learning.
- Fine tune pretrained CNN models.
- Compare classification performance of different architectures.

2. Learning Outcomes

After completing this experiment, the student will be able to:

1. Explain modern CNN architectures.
2. Differentiate LeNet, AlexNet, VGG16, GoogleNet and ResNet.
3. Implement transfer learning.
4. Fine tune pretrained CNN models.
5. Compare CNN architectures using performance metrics.

3. Introduction

Deep Convolutional Neural Networks have become important models for computer vision because they automatically learn hierarchical features from images. The evolution of CNNs has focused on improving classification accuracy while addressing computational cost, optimization difficulty and the need to learn increasingly complex visual representations.

The progression of CNN architectures began with LeNet-5 and evolved through AlexNet, VGGNet, GoogleNet and ResNet. Each architecture introduced important ideas such as ReLU activations, deeper networks, multi-scale feature extraction and residual learning.

A further development in deep learning is transfer learning. Instead of training a deep CNN completely from randomly initialized parameters, a model pretrained on a large dataset such as ImageNet can be reused as a feature extractor. Only the final classification layers need to be adapted initially, and selected pretrained layers can subsequently be fine tuned for the target dataset.

In this experiment, CIFAR-10 is used as the target classification dataset. The main experiment uses MobileNetV2 pretrained on ImageNet, followed by a comparison of additional transfer-learning and architectural experiments using VGG16, ResNet50, LeNet and AlexNet.

4. Evolution of CNN Architectures

Model	Year	Major Contribution
LeNet-5	1998	First practical convolutional neural network
AlexNet	2012	ReLU activation, Dropout and GPU training
VGG16	2014	Uniform 3×3 convolution filters
GoogleNet	2014	Inception modules for multi-scale feature extraction
ResNet	2015	Residual learning using skip connections

5. LeNet-5

LeNet-5 was proposed by Yann LeCun and collaborators for handwritten digit recognition. It consists of convolutional layers, average pooling layers and fully connected layers. The network demonstrated that useful image representations could be learned automatically rather than requiring handcrafted features.

Advantages:

- Small number of parameters.
- Fast training.
- Simple architecture.
- Suitable for relatively simple image-classification tasks.

6. AlexNet

AlexNet produced a major breakthrough in image classification in 2012. It demonstrated that deep convolutional networks trained using GPUs could significantly improve ImageNet classification performance.

Major innovations:

- ReLU activation.
- Dropout regularization.
- GPU-based training.
- Data augmentation.

7. VGG16

VGG16 demonstrated the effectiveness of increasing network depth while using small 3×3 convolution filters throughout the network.

Multiple small convolution layers can produce a large effective receptive field while introducing additional nonlinearities and maintaining a relatively uniform architecture.

Advantages:

- Strong feature extraction capability.
- Simple and uniform architecture.
- Effective pretrained feature extractor.
- Useful for transfer learning.

8. Comparison of Architectures

Architecture	Depth	Approx. Parameters	Main Contribution
LeNet-5	7	60K	Early practical CNN
AlexNet	8	61M	ReLU + Dropout + GPU training
VGG16	16	138M	Deep 3×3 convolutional filters
GoogleNet	22	6.8M	Inception modules
ResNet50	50	25.6M	Residual learning

The parameter counts above refer to the conventional architectures. The actual implementations used in this experiment have different parameter counts because the original ImageNet classification heads were removed in the transfer-learning models.

9. GoogleNet (Inception Network)

GoogleNet introduced the Inception module, which processes information using multiple operations in parallel. Typical Inception modules use 1×1 , 3×3 and 5×5 convolutions together with pooling.

The outputs are concatenated to form a multi-scale feature representation. 1×1 convolutions can also reduce the number of channels before larger convolutions, thereby reducing computation.

Advantages:

- Multi-scale feature extraction.
- Efficient use of computation.
- Fewer parameters than a comparably wide VGG-style network.
- Effective representation of objects at different scales.

10. ResNet

Increasing the depth of CNNs may make optimization difficult because of gradient-related problems. ResNet addresses this using residual or skip connections.

Instead of directly learning

$$H(x),$$

a residual block learns

$$F(x) = H(x) - x.$$

Therefore,

$$\text{Output} = F(x) + x.$$

Here, $F(x)$ is the residual mapping and x is the identity mapping.

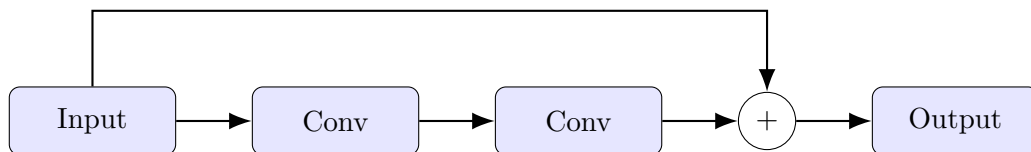


Figure 1: Residual Block in ResNet

Advantages:

- Provides a direct path for gradient propagation.

- Makes very deep networks easier to optimize.
- Enables architectures such as ResNet50 to be trained effectively.
- Provides strong hierarchical feature representations.

11. Dilated Convolution

Dilated convolution, also called atrous convolution, increases the receptive field without increasing the number of kernel parameters proportionally.

For a standard convolution,

$$D = 1,$$

whereas for a dilated convolution,

$$D > 1.$$

The dilation rate determines the spacing between kernel elements.

Applications:

- Semantic segmentation.
- Medical image analysis.
- Satellite imagery.
- Object localization.

12. Transpose Convolution

Transpose convolution increases the spatial dimensions of feature maps through learnable upsampling. Unlike ordinary pooling, which reduces spatial dimensions, transpose convolution is commonly used in decoder architectures.

Applications:

- Image super-resolution.
- Autoencoders.
- GANs.
- Image generation.
- Semantic segmentation.

13. Transfer Learning

Transfer learning utilizes a pretrained CNN model that has already learned useful visual features from a large dataset such as ImageNet.

The general workflow is:

1. Load a pretrained model.
2. Remove its original classifier.
3. Freeze the convolutional base.
4. Add a new classification head.
5. Train the new classifier.
6. Fine tune selected layers when necessary.

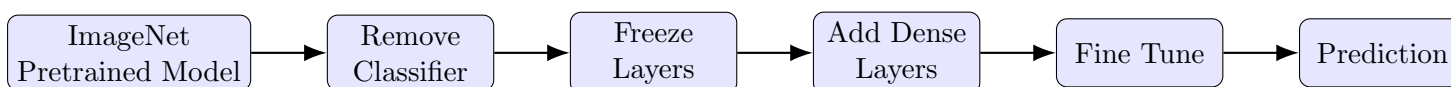


Figure 2: Transfer Learning Workflow

Advantages:

- Faster convergence.

- Higher accuracy on relatively small datasets.
- Reduced training time.
- Lower number of trainable parameters.

14. Dataset

The dataset used for this experiment is CIFAR-10.

- Training images: 50,000.
- Testing images: 10,000.
- Number of classes: 10.
- Image size: $32 \times 32 \times 3$.
- Image type: RGB.

The ten classes are:

Airplane, Automobile, Bird, Cat, Deer, Dog, Frog, Horse, Ship, Truck.

For the additional exercises, smaller stratified subsets were extracted from the actual CIFAR-10 dataset to reduce execution time:

Subset	Number of Images
Training	5,000
Validation	1,000
Testing	1,000

The additional experiments therefore use real CIFAR-10 images rather than synthetic or randomly generated data.

15. Experimental Procedure

Task 1: Dataset Preparation

The CIFAR-10 dataset was loaded using TensorFlow/Keras.

For the main MobileNetV2 experiment, the original training and testing sets were used. Pixel values were converted from integer RGB values to floating-point values in the range $[0, 1]$.

The original dataset dimensions were:

$$X_{\text{train}} = (50000, 32, 32, 3),$$

$$X_{\text{test}} = (10000, 32, 32, 3).$$

For the additional exercises, a stratified subset was extracted:

$$X_{\text{train}} = (5000, 32, 32, 3),$$

$$X_{\text{validation}} = (1000, 32, 32, 3),$$

$$X_{\text{test}} = (1000, 32, 32, 3).$$

Ten sample CIFAR-10 images were also displayed to verify the dataset contents.

Task 2: Transfer Learning using MobileNetV2

MobileNetV2 was selected as the main pretrained CNN. ImageNet pretrained weights were loaded using `include_top=False`.

Because CIFAR-10 images are 32×32 , they were resized to

$$96 \times 96 \times 3.$$

The resized images were then transformed approximately to the range $[-1, 1]$, which is the expected input range for MobileNetV2.

The convolutional base was frozen and the following classification head was added:

$$\text{GlobalAveragePooling2D} \rightarrow \text{Dense}(128, \text{ReLU}) \rightarrow \text{Dropout}(0.3) \rightarrow \text{Dense}(10, \text{Softmax}).$$

The resulting model summary showed:

Total Parameters	2,423,242
Trainable Parameters	165,258
Non-trainable Parameters	2,257,984

Thus, only a small fraction of the total model parameters needed to be optimized during the initial transfer-learning phase.

Task 3: Model Training

The initial MobileNetV2 training configuration was:

Optimizer	Adam
Learning Rate	0.001
Batch Size	32
Epochs	15
Loss Function	Categorical Cross Entropy
Evaluation Metric	Accuracy
Validation Split	20%

The model's training accuracy increased from 79.66% in the first epoch to 93.93% after 15 epochs.

Validation accuracy reached a maximum of 87.12% at epoch 11. The validation loss reached its minimum of 0.3912 at epoch 5 and subsequently increased, while training loss continued to decrease. This indicates that overfitting began to appear during the later epochs.

The frozen-base MobileNetV2 model obtained:

Test Accuracy	86.24%
Test Loss	0.5257

Task 4: Fine Tuning

After the initial transfer-learning stage, the MobileNetV2 base was partially unfrozen.

The last 20 layers of the pretrained base were made trainable while the earlier layers remained frozen.

The learning rate was reduced from

$$0.001 \rightarrow 0.0001$$

to avoid excessively large changes to the pretrained weights.

The fine-tuning stage lasted seven additional epochs.

Training accuracy increased from 83.11% at the beginning of the fine-tuning stage to 95.67% at its end.

The highest validation accuracy during fine tuning was 87.84%.

Stage	Test Accuracy
Before Fine Tuning	86.24%
After Fine Tuning	87.55%

The final test loss decreased from 0.5257 to 0.5183.

Task 5: Model Evaluation

The final fine-tuned MobileNetV2 model was evaluated using accuracy, precision, recall, F1-score, classification report and a confusion matrix.

Metric	Value
Accuracy	0.8755
Weighted Precision	0.8801
Weighted Recall	0.8755
Weighted F1-score	0.8752
Test Loss	0.5183
Total Parameters	2,423,242

Class-wise Classification Report:

Class	Precision	Recall	F1	Support
Airplane	0.85	0.93	0.89	1000
Automobile	0.94	0.94	0.94	1000
Bird	0.81	0.89	0.85	1000
Cat	0.77	0.80	0.78	1000
Deer	0.80	0.88	0.84	1000
Dog	0.94	0.70	0.80	1000
Frog	0.94	0.86	0.90	1000
Horse	0.88	0.92	0.90	1000
Ship	0.91	0.94	0.92	1000
Truck	0.96	0.90	0.93	1000

The Dog class had the lowest recall at 0.70, indicating that many actual dog images were assigned to other classes. The strongest class-wise F1-scores were obtained for Automobile (0.94), Truck (0.93) and Ship (0.92).

16. Hyperparameter Study

The experiment specification lists the following possible hyperparameters:

Hyperparameter	Values
Learning Rate	0.001, 0.0001
Batch Size	16, 32, 64
Epochs	10, 20
Optimizer	Adam, SGD
Dense Units	128, 256
Frozen Layers	All, Partial

The notebook did not perform a complete Cartesian grid search over all of these combinations. Instead, the following comparisons were implemented directly:

1. Adam versus SGD.
2. Fully frozen VGG16 versus partially fine-tuned VGG16.
3. VGG16 versus ResNet50 transfer learning.
4. LeNet-5 versus AlexNet versus ResNet50.

The additional experiments use the same stratified CIFAR-10 subsets so that the results are more directly comparable.

17. Mandatory Plots

The notebook generated the following plots:

1. Sample CIFAR-10 images.
2. Training and validation accuracy.
3. Training and validation loss.
4. Confusion matrix.
5. Accuracy before and after fine tuning.
6. Misclassified CIFAR-10 images.
7. Training histories and confusion matrices for the additional experiments.

17.1 Sample CIFAR-10 Images

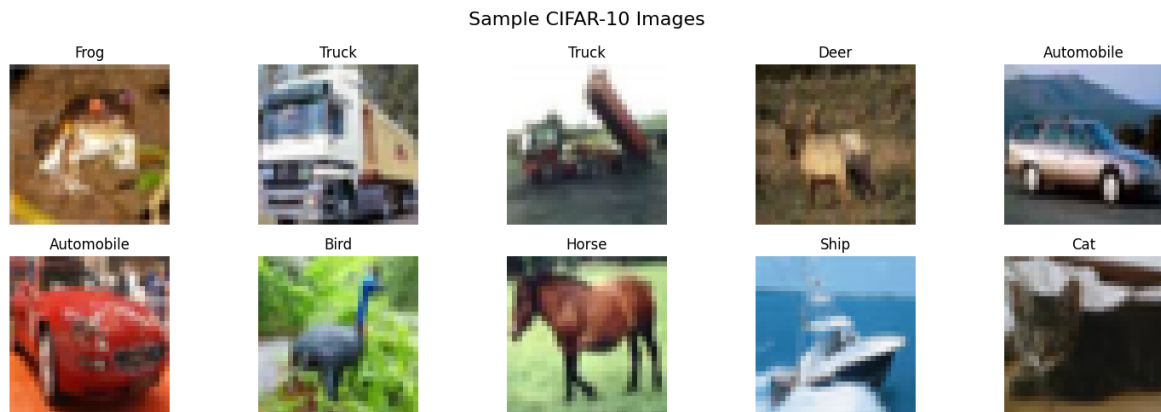


Figure 1: Sample CIFAR-10 images

Inference: The displayed samples represent the ten CIFAR-10 classes and show the variety in object appearance, pose and background. The low 32×32 resolution makes the classification task challenging.

17.2 Training and Validation Accuracy

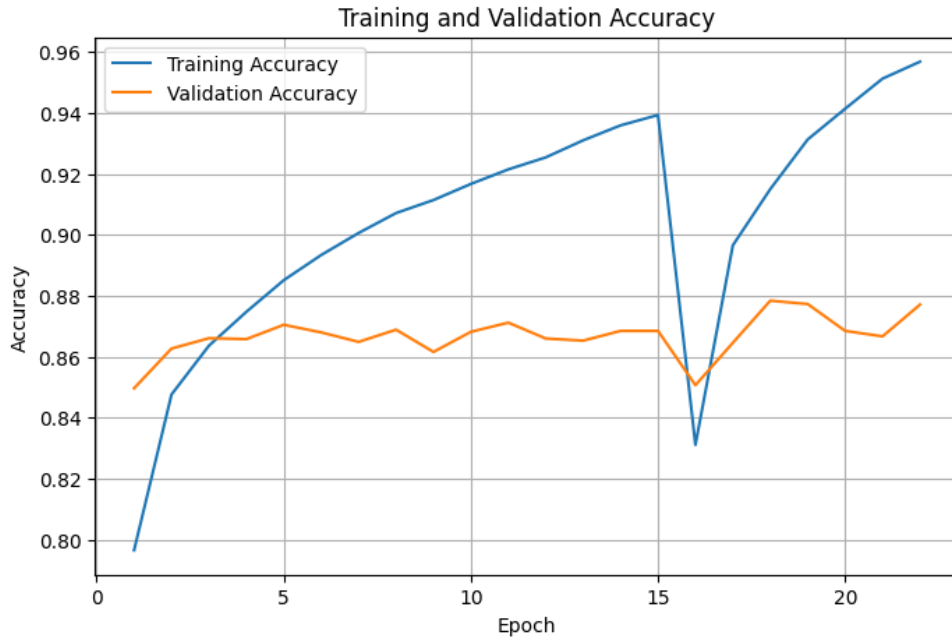


Figure 2: Training and validation accuracy

Inference: Training accuracy rises substantially throughout the experiment, reaching 95.67% after fine tuning. Validation accuracy remains around the mid-to-high 80% range. The increasing gap between training and validation accuracy indicates overfitting in later epochs.

17.3 Training and Validation Loss

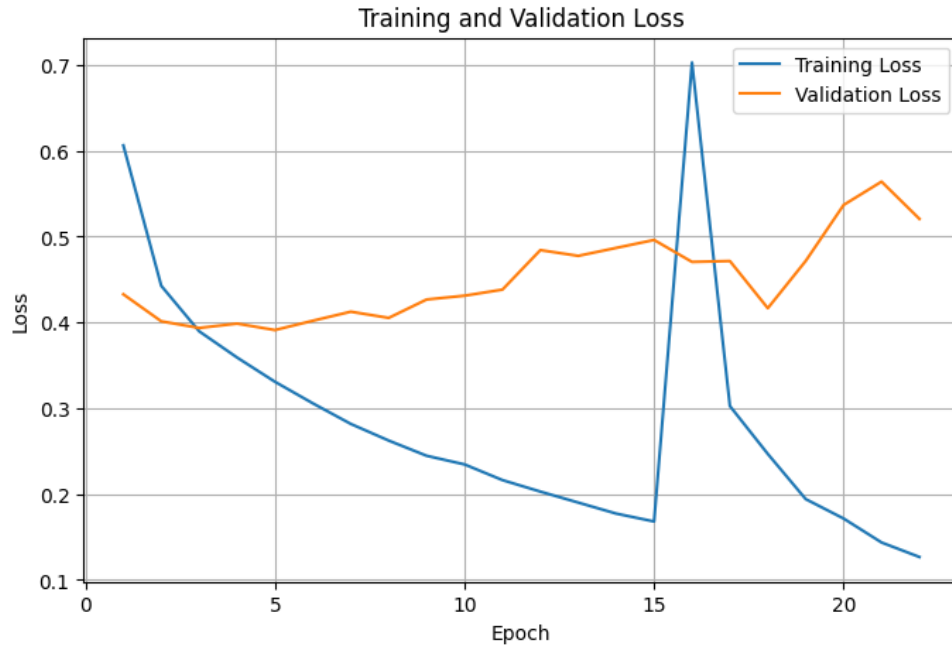


Figure 3: Training and validation loss

Inference: Training loss decreases throughout training, while validation loss reaches a minimum earlier and then increases. This indicates that continued optimization improves fit to the training set more than it improves generalization.

17.4 Confusion Matrix

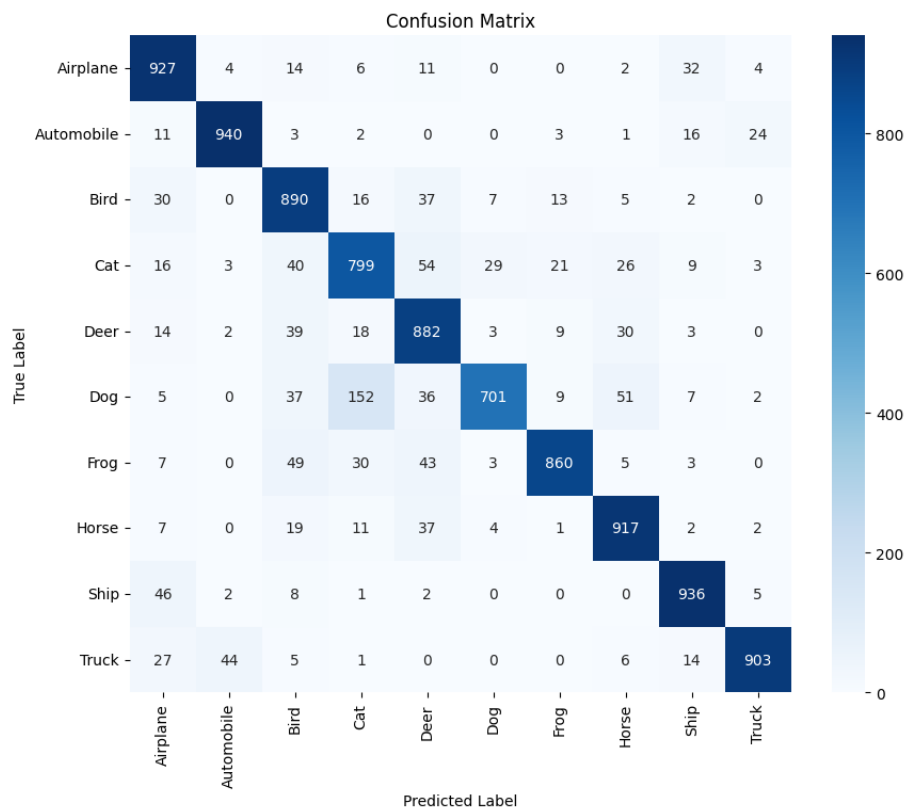


Figure 4: Confusion matrix for the final MobileNetV2 model

Inference: The confusion matrix shows strong diagonal performance for most classes. Dog produces more off-diagonal errors, particularly with visually similar animal categories such as Cat.

17.5 Accuracy Before and After Fine Tuning

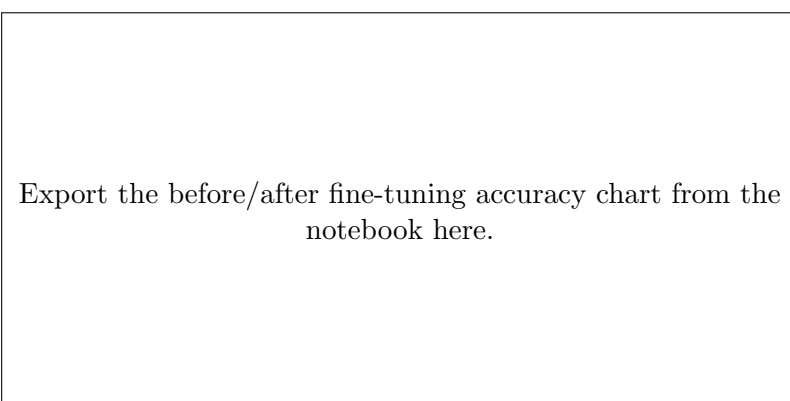


Figure 5: Test accuracy before and after fine tuning

Inference: Fine tuning improves the test accuracy from 86.24% to 87.55%. The 1.31 percentage-point improvement shows that adapting the upper layers of the pretrained backbone is beneficial for CIFAR-10.

17.6 Misclassified Images

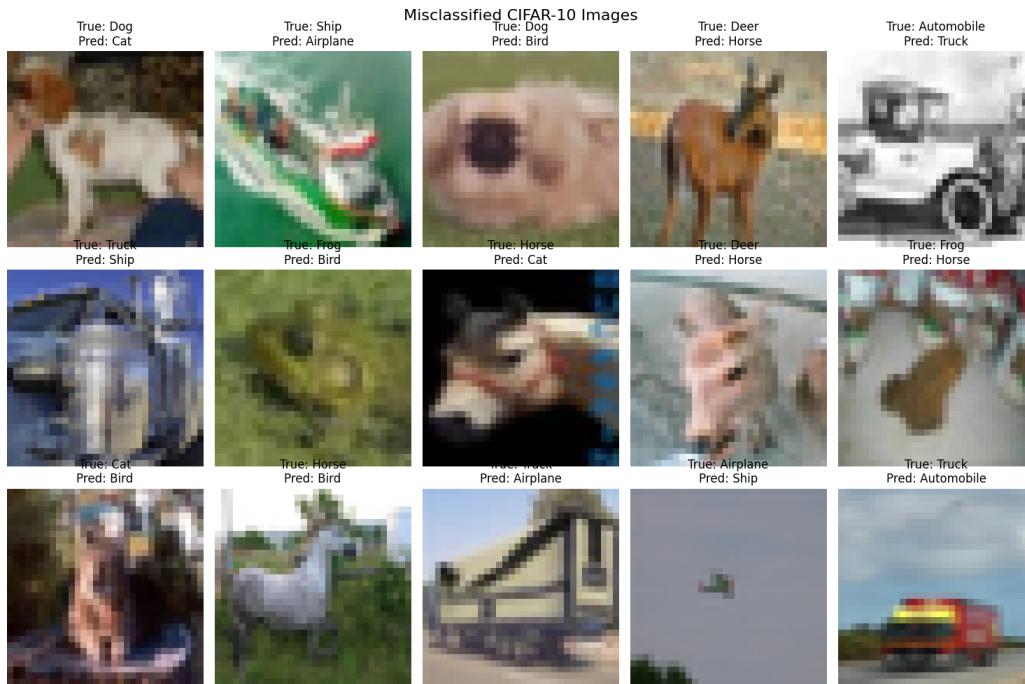


Figure 6: Misclassified CIFAR-10 images

Inference: The misclassified examples generally correspond to visually ambiguous images or classes with similar visual characteristics. The low resolution of CIFAR-10 contributes to this difficulty.

18. Results

18.1 Main MobileNetV2 Experiment

Metric	Value
Final Training Accuracy	95.67%
Test Accuracy Before Fine Tuning	86.24%
Final Test Accuracy	87.55%
Weighted Precision	0.8801
Weighted Recall	0.8755
Weighted F1-score	0.8752
Test Loss	0.5183
Total Parameters	2,423,242
Trainable Parameters Before Fine Tuning	165,258

18.2 Additional Exercise 1: VGG16 Transfer Learning

A VGG16 model pretrained on ImageNet was used with `include_top=False`. The convolutional base was frozen and followed by:

GlobalAveragePooling2D → Dense(128, ReLU) → Dropout(0.3) → Dense(10, Softmax).

The experiment used the 5,000-image training subset and 1,000-image validation and testing subsets. Images were resized to 96×96 and the VGG16 preprocessing function was applied. The model contained 14,781,642 total parameters, of which 66,954 were trainable.

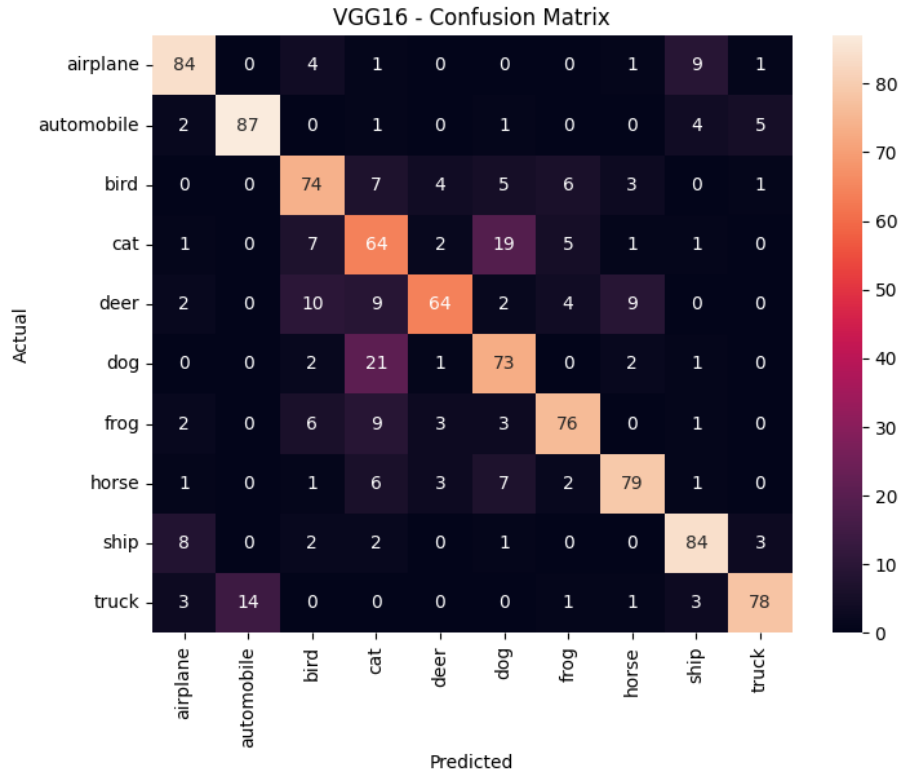


Figure: VGG16 Confusion Matrix

Inference: The VGG16 confusion matrix shows relatively strong classification for vehicle classes such as Automobile, Ship and Truck. Greater confusion occurs among animal classes, particularly Cat, Deer and Dog, which have more off-diagonal predictions.

Metric	VGG16 Result
Test Accuracy	76.30%
Weighted Precision	0.7723
Weighted Recall	0.7630
Weighted F1-score	0.7652
Total Parameters	14,781,642
Training Time	88.01 s

The final VGG16 training accuracy was approximately 92.46%, while validation accuracy was approximately 79.70%. The difference suggests the beginning of overfitting despite the frozen convolutional base.

18.3 Additional Exercise 2: ResNet50 Transfer Learning

ResNet50 was loaded with ImageNet pretrained weights and `include_top=False`. The pretrained convolutional base was frozen and a new classification head was attached.

The model contained 23,851,274 total parameters. The base ResNet50 feature extractor contributed 23,587,712 parameters and the new classification head contributed the remaining parameters.

The model was trained for 10 epochs using Adam with learning rate 0.001.

Metric	ResNet50 Result
Test Accuracy	81.10%
Weighted Precision	0.8131
Weighted Recall	0.8110
Weighted F1-score	0.8114
Total Parameters	23,851,274
Training Time	66.58 s

The final training accuracy was 96.98%, compared with validation accuracy of 84.40%. This indicates a notable training–validation gap, showing some overfitting even though the pretrained convolutional base was frozen.

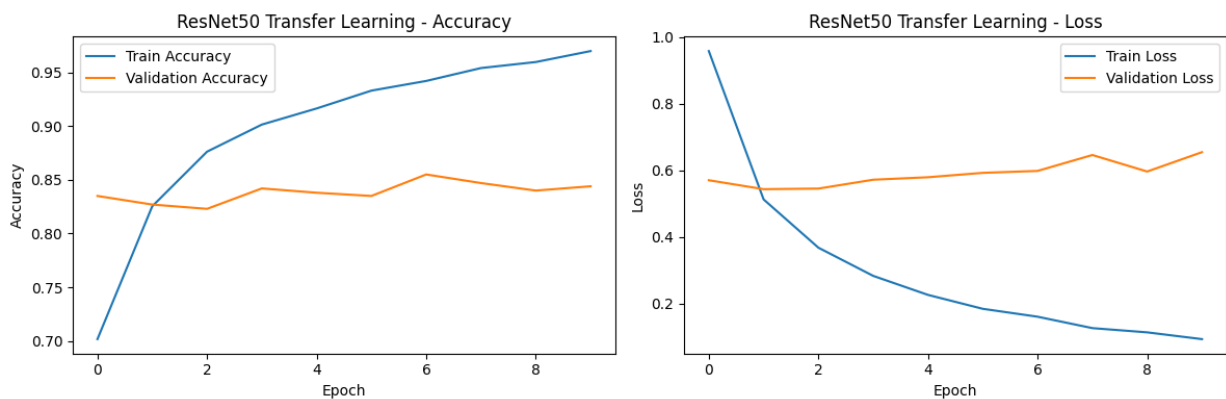


Figure: ResNet50 Transfer Learning – Training and Validation Accuracy/Loss

Inference: The ResNet50 training accuracy increases steadily from approximately 70% to 97%, whereas validation accuracy remains around 82–85%. Training loss decreases continuously, while validation loss remains considerably higher and fluctuates, indicating overfitting as training progresses.

18.4 Additional Exercise 3: Adam versus SGD

VGG16 transfer learning was used to compare the Adam and SGD optimizers.

The Adam run used 10 epochs, while the SGD run used 2 epochs. Therefore, the comparison reflects the actual notebook runs but is not a perfectly controlled comparison of the two optimizers because the number of training epochs differs.

Optimizer	Epochs	Accuracy	Precision	Recall	F1
Adam	10	77.30%	0.7727	0.7730	0.7724
SGD	2	72.00%	0.7277	0.7200	0.7211

The measured training times were:

Adam Training Time	90.64 s
SGD Training Time	29.58 s

Adam achieved the higher observed accuracy in the notebook. However, SGD was evaluated after only two epochs, so the result should not be interpreted as a definitive optimizer benchmark.

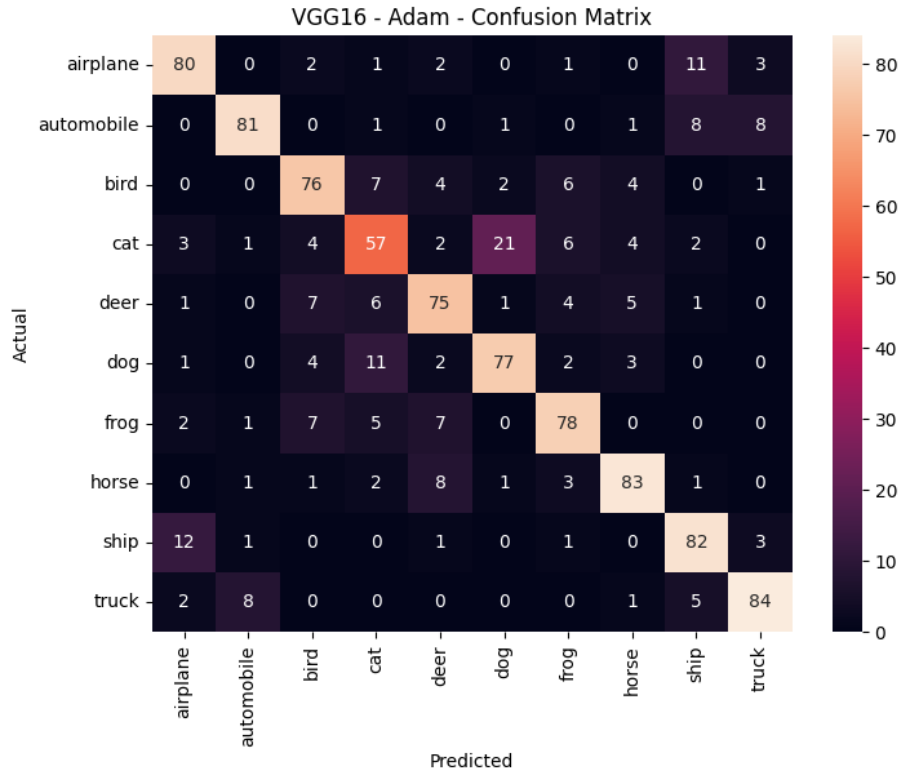


Figure: VGG16 with Adam Optimizer – Confusion Matrix

Inference: The Adam-trained VGG16 model gives stronger diagonal values than the SGD run for several classes. In particular, Automobile, Frog, Horse, Ship and Truck are classified relatively well, while Cat and Dog remain more challenging.

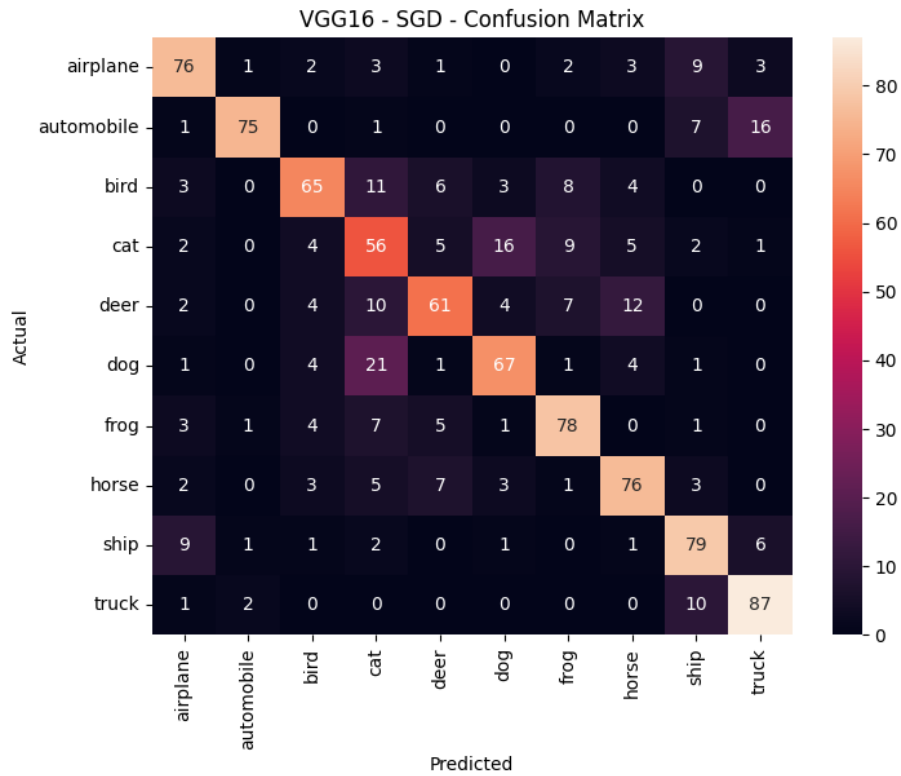


Figure: VGG16 with SGD Optimizer – Confusion Matrix

Inference: The SGD-trained VGG16 model shows moderate classification performance, with rea-

sonable diagonal values for several classes. However, the animal classes exhibit noticeable confusion, particularly between Cat, Dog, Deer and Horse, indicating that the model has not learned the class boundaries as effectively as the fine-tuned model.

18.5 Additional Exercise 4: Frozen Layers versus Fine Tuning

The VGG16 model was first evaluated with its entire pretrained convolutional base frozen.

The first phase produced:

Stage	Accuracy	Precision	Recall	F1
Frozen Base	76.40%	0.7629	0.7640	0.7619
Fine Tuned	83.70%	0.8489	0.8370	0.8382

During fine tuning, the final convolutional block of VGG16 was unfrozen and the learning rate was reduced to 0.0001.

The measured improvement was:

$$83.70 - 76.40 = \boxed{7.30 \text{ percentage points}}.$$

This represents a substantially larger improvement than the 1.31 percentage-point improvement observed when fine tuning MobileNetV2.

The result demonstrates that the generic features produced by a frozen pretrained network can be further adapted to the target dataset.

18.6 Additional Exercise 5: Precision, Recall and F1-score

The final VGG16 frozen-base evaluation used the following overall metrics:

Accuracy	0.7630
Weighted Precision	0.7723
Weighted Recall	0.7630
Weighted F1-score	0.7652

The class-wise results were:

Class	Precision	Recall	F1
Airplane	0.82	0.84	0.83
Automobile	0.86	0.87	0.87
Bird	0.70	0.74	0.72
Cat	0.53	0.64	0.58
Deer	0.83	0.64	0.72
Dog	0.66	0.73	0.69
Frog	0.81	0.76	0.78
Horse	0.82	0.79	0.81
Ship	0.81	0.84	0.82
Truck	0.89	0.78	0.83

Cat had the weakest F1-score, at 0.58, while Automobile and Truck showed stronger performance.

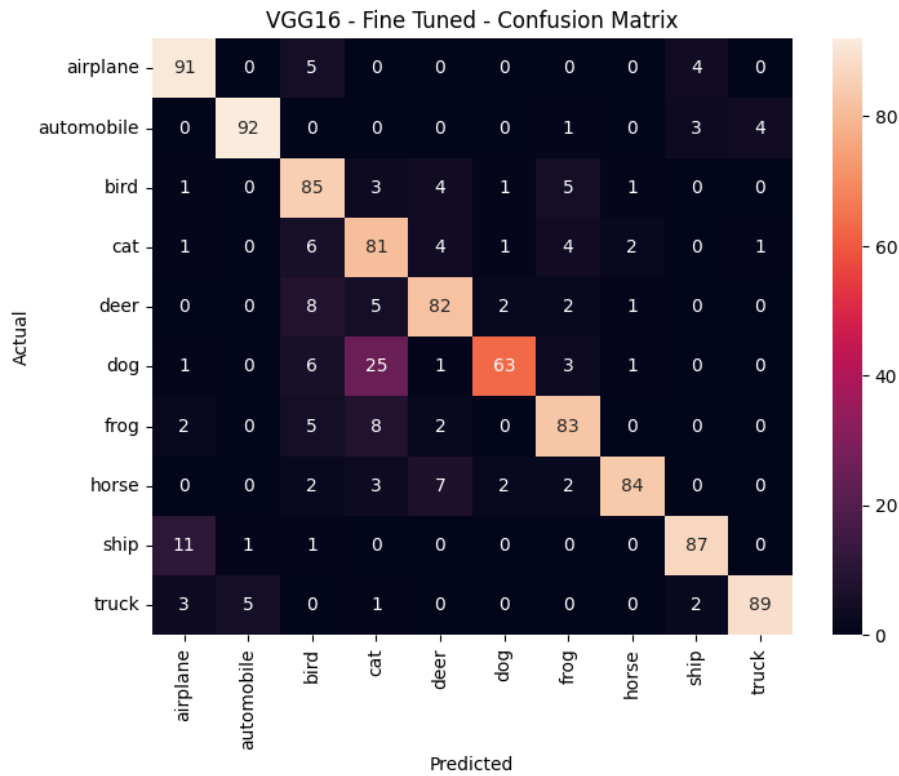


Figure: VGG16 Fine-Tuned Confusion Matrix

Inference: Fine tuning substantially increases the diagonal values across the classes compared with the frozen VGG16 model. Most classes are now classified correctly with high frequency, although Dog still shows some confusion with Cat and other animal categories.

18.7 Additional Exercise 6: LeNet, AlexNet and ResNet50

LeNet-5 and AlexNet were trained from scratch on the same actual CIFAR-10 subset. The ResNet50 result was reused from Additional Exercise 2 instead of retraining the model a second time.

The measured results were:

Model	Parameters	Accuracy	Training Time
LeNet-5	62,006	47.70%	11.65 s
AlexNet	833,738	54.50%	17.36 s
ResNet50	23,851,274	81.10%	66.58 s

The corresponding weighted metrics were:

Model	Precision	Recall	F1-score
LeNet-5	0.4905	0.4770	0.4757
AlexNet	0.5573	0.5450	0.5420
ResNet50	0.8131	0.8110	0.8114

LeNet-5 produced the lowest accuracy, followed by AlexNet, while ResNet50 achieved the strongest performance.

It is important to note that LeNet-5 and AlexNet were trained from random initialization, whereas ResNet50 used ImageNet pretrained weights. Therefore, the comparison demonstrates both architectural differences and the practical benefit of transfer learning.

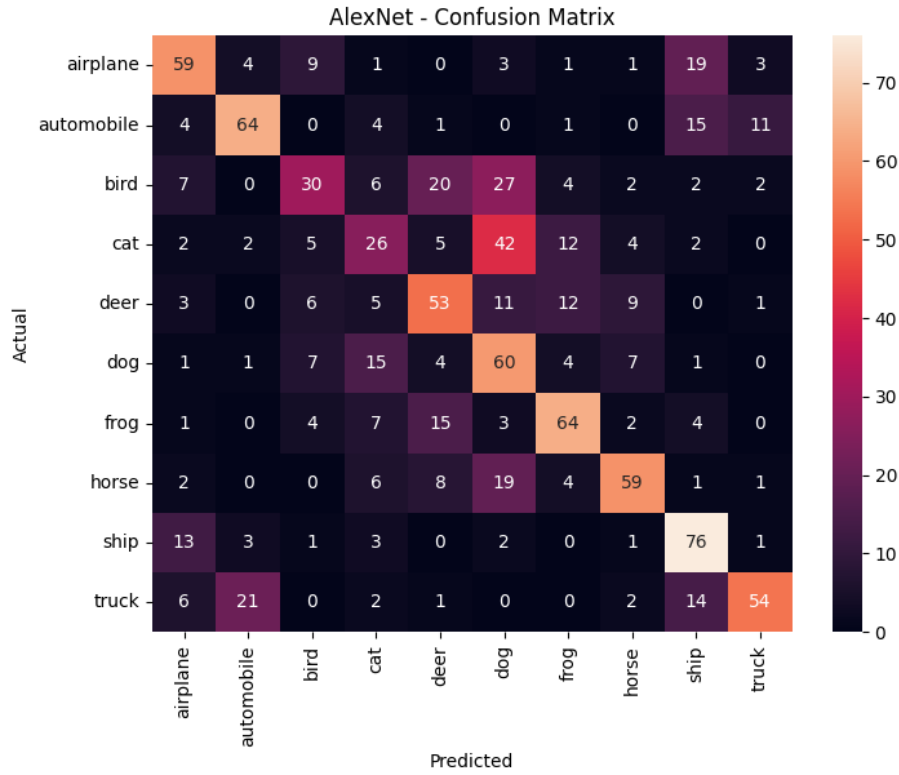


Figure: AlexNet Confusion Matrix

Inference: AlexNet achieves its strongest predictions for classes such as Ship, Frog, Horse and Dog. Significant confusion is visible between visually similar classes, especially Bird–Deer, Cat–Dog and Automobile–Truck, which limits the overall accuracy of the model.

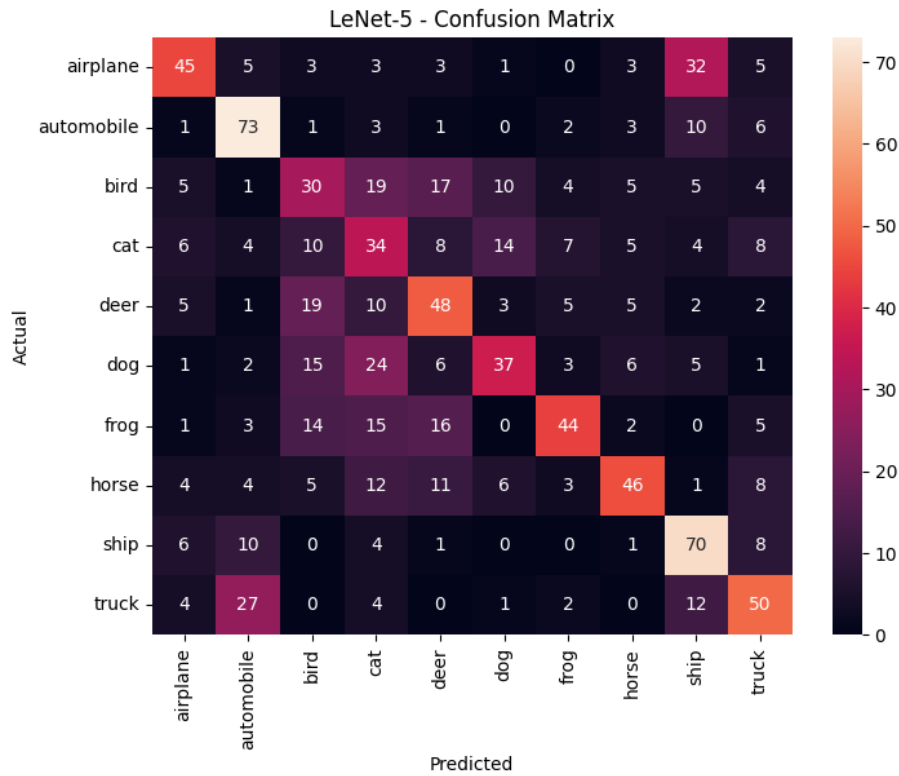


Figure: LeNet-5 Confusion Matrix

Inference: LeNet-5 has substantially more off-diagonal predictions than the deeper architectures.

Automobile and Ship are classified relatively well, but there is considerable confusion among Bird, Cat, Deer, Dog, Frog and Horse, reflecting the limited capacity of the shallow network.

18.8 Additional Exercise Summary

Experiment	Accuracy	Precision	Recall	F1
VGG16	76.30%	0.7723	0.7630	0.7652
ResNet50	81.10%	0.8131	0.8110	0.8114
VGG16 + Adam	77.30%	0.7727	0.7730	0.7724
VGG16 + SGD	72.00%	0.7277	0.7200	0.7211
VGG16 Frozen	76.40%	0.7629	0.7640	0.7619
VGG16 Fine Tuned	83.70%	0.8489	0.8370	0.8382
LeNet-5	47.70%	0.4905	0.4770	0.4757
AlexNet	54.50%	0.5573	0.5450	0.5420
ResNet50	81.10%	0.8131	0.8110	0.8114
MobileNetV2 Fine Tuned	87.55%	0.8801	0.8755	0.8752

19. Discussion Questions

1. Why is AlexNet considered a breakthrough in deep learning?
2. Why does VGG16 use only 3×3 convolution filters?
3. Explain the advantages of the Inception module.
4. What is the purpose of residual learning?
5. Differentiate LeNet and ResNet.
6. What is Transfer Learning?
7. Why is fine tuning required?
8. Explain the difference between dilated convolution and transpose convolution.
9. Why do pretrained models converge faster?
10. Compare the computational complexity of LeNet and ResNet.

Answers

1. Why is AlexNet considered a breakthrough in deep learning?

AlexNet demonstrated that deep convolutional neural networks trained with GPUs, ReLU activations and Dropout could achieve greatly improved image-classification performance. Its success showed the practical benefit of deep CNNs for large-scale computer vision.

2. Why does VGG16 use only 3×3 convolution filters?

Stacking 3×3 convolution filters creates a large effective receptive field while keeping the architecture simple. Multiple convolution layers also provide additional nonlinearities, increasing the expressive capability of the network.

3. Explain the advantages of the Inception module.

The Inception module performs multiple operations in parallel, allowing the network to extract features at different spatial scales. The use of 1×1 convolutions can reduce the number of channels before larger convolutions and therefore reduce computation.

4. What is the purpose of residual learning?

Residual learning allows a network to learn a residual mapping instead of directly learning the complete transformation. Skip connections provide a direct path through the network, helping gradients propagate through deep architectures.

5. Differentiate LeNet and ResNet.

LeNet is a comparatively small and shallow CNN originally designed for handwritten digit recognition. ResNet is much deeper and uses residual connections to make the optimization of deep archi-

tructures practical. In the current experiment, the implemented LeNet model had 62,006 parameters, whereas the implemented ResNet50 model had 23,851,274 parameters.

6. What is Transfer Learning?

Transfer learning reuses knowledge learned by a model on a large source dataset for a different target task. In this experiment, CNNs pretrained on ImageNet were adapted for CIFAR-10 classification.

7. Why is fine tuning required?

A completely frozen pretrained network acts as a fixed feature extractor. Fine tuning allows higher-level features to adapt to the target dataset. In the VGG16 experiment, fine tuning improved test accuracy from 76.40% to 83.70%.

8. Explain the difference between dilated convolution and transpose convolution.

Dilated convolution increases the receptive field by inserting spaces between kernel elements while retaining the spatial resolution of the feature map. Transpose convolution performs learnable upsampling and increases the spatial dimensions of the feature map.

9. Why do pretrained models converge faster?

Pretrained models already contain visual features learned from a large dataset. Therefore, optimization begins from useful representations instead of random weights. Only the task-specific classifier and, optionally, selected backbone layers need to be adapted.

10. Compare the computational complexity of LeNet and ResNet.

LeNet is significantly smaller than ResNet50. In this experiment, LeNet had 62,006 parameters and a training time of 11.65 seconds, whereas ResNet50 had 23,851,274 parameters and a training time of 66.58 seconds. ResNet50 therefore provides substantially greater representational capacity at a higher computational cost.

20. Additional Exercises

1. Implement transfer learning using VGG16.
2. Repeat the experiment using ResNet50.
3. Compare Adam and SGD optimizers.
4. Compare training with frozen layers and fine tuning.
5. Evaluate the model using Precision, Recall and F1-score.
6. Compare the performance of LeNet, AlexNet and ResNet.

The additional exercises were completed in the accompanying notebook using a smaller stratified subset of the actual CIFAR-10 dataset.

21. Source Code

The accompanying Jupyter notebook contains the complete implementation used for the experiment and additional exercises.

The implementation includes:

- CIFAR-10 dataset loading.
- Normalization and label encoding.
- MobileNetV2 transfer learning.
- Initial frozen-base training.
- MobileNetV2 fine tuning.
- VGG16 transfer learning.
- ResNet50 transfer learning.
- Adam versus SGD comparison.
- Frozen VGG16 versus fine-tuned VGG16.
- LeNet-5 implementation and training.
- AlexNet implementation and training.
- Precision, recall and F1-score calculation.

- Classification reports.
- Confusion matrices.
- Training and validation curves.
- Misclassified-image visualization.

For the additional experiments, the notebook uses a stratified subset of 5,000 training images, 1,000 validation images and 1,000 testing images from CIFAR-10 to reduce execution time while retaining real dataset samples.

22. Conclusion

This experiment demonstrated the evolution of convolutional neural network architectures and the practical implementation of transfer learning for image classification. The main experiment utilized MobileNetV2, pretrained on the ImageNet dataset, as an extractor for the CIFAR-10 dataset. For the transfer learning process, the convolutional base was frozen, thus resulting in only 165,258 trainable parameters out of 2,423,242 overall parameters. The final test accuracy of the frozen-base experiment was 86.24% on the CIFAR-10 test set after 15 epochs. The fine-tuning process involved unfreezing the last 20 layers and reducing the learning rate to 0.0001, which increased the test accuracy by 1.31% to 87.55%. Thus, the final weighted F1-score was 0.8752.

The additional VGG16 experiment reported a test accuracy of 76.30% for the frozen base model, while the ResNet50 architecture achieved 81.10% using the same general transfer learning procedure. The VGG16 optimizer comparison reported a test accuracy of 77.30% for the Adam optimizer and 72.00% for the SGD optimizer in the recorded experiment runs. While the difference between the performance of the optimizers may suggest a significant change in model performance, the comparison is unfair since the VGG16 fine-tuned with SGD only used 10 epochs as compared to 15 epochs for the other experiments.

The comparison between the frozen and fine-tuned VGG16 architectures demonstrated the value of task-specific adaptation, reflected in an increase in test accuracy from 76.40% to 83.70%. Thus, the concept of transfer learning was demonstrated using two different CNN architectures. In the architecture comparison experiment, the LeNet-5 CNN achieved 47.70% test accuracy, while AlexNet achieved 54.50%, compared to 81.10% for the pretrained ResNet50 architecture. Therefore, both the CNN depth and the use of transfer learning significantly contribute to the final model performance.

In conclusion, this series of experiments demonstrated that transfer learning can enable high image-classification performance without training a CNN from scratch. The test accuracy can be improved by fine-tuning certain layers of a pretrained convolutional neural network to adapt it to the target domain. Among the models directly compared in the experiments, the fine-tuned MobileNetV2 architecture achieved the highest test accuracy of 87.55%.

23. References

1. Yann LeCun, Leon Bottou, Yoshua Bengio and Patrick Haffner, *Gradient-Based Learning Applied to Document Recognition*, Proceedings of the IEEE, 1998.
2. Alex Krizhevsky, Ilya Sutskever and Geoffrey Hinton, *ImageNet Classification with Deep Convolutional Neural Networks*, NeurIPS, 2012.
3. Karen Simonyan and Andrew Zisserman, *Very Deep Convolutional Networks for Large-Scale Image Recognition*, ICLR, 2015.
4. Christian Szegedy et al., *Going Deeper with Convolutions*, CVPR, 2015.
5. Kaiming He, Xiangyu Zhang, Shaoqing Ren and Jian Sun, *Deep Residual Learning for Image Recognition*, CVPR, 2016.
6. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
7. TensorFlow Documentation: <https://www.tensorflow.org>
8. Keras Documentation: <https://keras.io>